Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 19/10/2026**

MINI PROJECT REPORT ON

"Smart Restaurant Management System: A Chatbot-Enabled Restaurant Ordering Application Delivered Through an Agile and DevOps Pipeline"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Heenal Dhanwani (Roll No. T1, 2401024)

Ishita Chimnani (Roll No. T1, 2401019)

SUBJECT

Agile Software Development and DevOps Lab

SUPERVISOR

Prof. Sanober Shaikh



CERTIFICATE

This is to certify that the mini project entitled **"Smart Restaurant Management System: A Chatbot-Enabled Restaurant Ordering Application Delivered Through an Agile and DevOps Pipeline"** is a bona fide work of **Heenal Dhanwani (Roll No. 2401024)** and **Ishita Chimnani (Roll No. 2401019)**, submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide **Head of Department**

Date: _____ Place: _____

ABSTRACT

Restaurants that take orders by hand lose time and make mistakes: menus are printed, orders are written on paper, totals are added by hand, and customers have no quick way of asking about a dish or about the progress of their order. The Smart Restaurant Management System addresses this with a web application in which a customer browses the menu by category, adds dishes to a cart, places an order with a name and table number, and asks a built-in Restaurant Assistant chatbot about the menu, prices, vegetarian dishes or the status of an order, while restaurant staff use a password-protected admin dashboard to track orders, change their status, view sales and manage the menu.

The application served as the vehicle for applying all seven experiments of the Agile Software Development & DevOps Lab to one product. The work was planned in Jira as a Scrum backlog and sprint and linked to GitHub, so that issue keys appear in branches, commits and pull requests. Every change was versioned with Git and merged through GitHub. The Flask application was packaged as a Docker image and, together with nginx, MySQL, Redis and phpMyAdmin, defined in a single Docker Compose file. Jenkins was installed to run Declarative and Scripted pipelines with Build, Test and Deploy stages, and a Jenkinsfile stored in GitHub allowed Jenkins to fetch and run the pipeline from source control. GitHub Actions runs the pytest tests on every push and pull request. Finally, Nagios Core running in a Docker container monitors a remote Windows host through the NSClient++ agent.

The work was divided between two members, each of whom performed part of every experiment on their own machine and GitHub account, and evidence in the form of command output and screenshots is included for every experiment. The project shows that Agile planning, version control, containers, continuous integration, automated deployment and monitoring are most useful when they operate together as one delivery flow: Jira → Git and GitHub → Docker and Docker Compose → Jenkins → Nagios.

Keywords: Agile, Scrum, Jira, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, GitHub Actions, Nagios, NSClient++, Flask, Python, SQLite, Chatbot

TABLE OF CONTENTS

Contents		Page Numbers
1.	Introduction	5
2.	Requirements Analysis	7
3.	System Design	9
4.	Application Implementation	11
5.	DevOps Implementation (Experiments 1–7)	15
6.	Testing	40
7.	Results and Discussion	41
8.	Conclusion and Future Scope	42
9.	References	42
10.	Appendix: Running the Project	43
11.	Lab Outcomes	43

1. Introduction

1.1 Background

A restaurant serves many tables at once, and every order passes through several hands: the customer, the waiter, the kitchen and the cashier. When orders are taken on paper, items are misheard, totals are calculated by hand, and customers wait for staff to answer simple questions about the menu, the prices or the progress of their food. A digital menu with ordering from the table, and an assistant that answers the common questions at once, reduces these delays and errors.

The Agile Software Development & DevOps Lab introduced, through seven separate experiments, the practices used to deliver software of this kind reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment was first performed in isolation on a small workload. This mini project applies all seven to one realistic application, the Smart Restaurant Management System, so that the way the tools fit together becomes visible, rather than each tool being seen on its own.

1.2 Problem Statement

To design, build and operate a web application that lets customers browse a restaurant menu, place orders from their table and ask a chatbot about dishes, prices and order status, and that lets staff manage orders and the menu, and to deliver it through an Agile, automated pipeline in which work is planned in Jira, versioned in Git and GitHub, packaged with Docker and Docker Compose, built and deployed by Jenkins, tested by GitHub Actions and observed by Nagios.

1.3 Objectives

1. Allow a customer to browse the menu by category, add dishes to a cart and place an order with a name and table number.
2. Provide a Restaurant Assistant chatbot that answers questions about the menu, prices, vegetarian dishes, recommendations and the status of an order, and an admin dashboard on which staff track orders, update their status and manage the menu.
3. Plan the work as a Jira Scrum backlog and sprint, with issue keys linking commits and pull requests to tickets.
4. Place the project under Git, publish it to GitHub, and practise branching, merging, conflict resolution, reset, revert and restore.
5. Package the application as a Docker image and run it as a container.
6. Run the application together with nginx, MySQL, Redis and phpMyAdmin from one Docker Compose file.
7. Install Jenkins, build Declarative and Scripted pipelines, and run a pipeline whose Jenkinsfile is fetched from GitHub.
8. Monitor a remote Windows host with Nagios and the NSClient++ agent.

1.4 Scope

The system serves one restaurant with a single menu, and orders are placed per table. Order totals are calculated on the server from the prices stored in the SQLite database, so the browser cannot change a price. The chatbot is rule-based: keywords and patterns in the question are matched to intents (greeting, menu, price, vegetarian, category, recommendation, opening hours and order status) and answered from the database, which keeps replies fast and needs no external service. The MySQL, Redis and phpMyAdmin services of the Compose experiment are demonstration services, as in the lab file; the application itself stores its data in SQLite. The deployment target is a developer machine running Docker; the same Dockerfile, Compose file and Jenkinsfile would apply to a college server. The Deploy stage of the Jenkins pipeline is demonstrated with echo steps, as in the lab experiments.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in the Smart Restaurant Management System	Section
1	Version control with Git	Repository smart-restaurant-system, feature branches, merge, merge conflict, reset, revert and restore	5.1
2	Containerisation with Docker	Dockerfile and image restaurant-manager:v1; 25 Docker commands	5.2
3	Docker Compose	compose-demo stack: web (nginx), app, database, redis, phpmyadmin	5.3
4	Jenkins CI pipeline	Declarative and Scripted pipelines with Build, Test and Deploy stages	5.4
5	Automated deployment	Jenkinsfile in GitHub, Pipeline script from SCM, GitHub hook trigger	5.5
6	Agile project lifecycle	Jira Scrum project SRT, sprint, GitHub for Jira, automation rules, GitHub Actions	5.6
7	Monitoring	Nagios Core in Docker monitoring a Windows host through NSClient++	5.7

Table 1.1: Lab experiments and where each is applied

1.6 Team Roles and Work Division

The group has two members. Every member performed part of every experiment on their own PC, with their own GitHub account, and took screenshots of their own commands. Heenal Dhanwani is the Foundation / Owner, who creates the project, the GitHub repository, the Jira space and the Jenkins and Nagios servers, writes the Dockerfile, the Jenkinsfile and the GitHub Actions workflow, develops the first feature branch, and reviews and merges the pull requests. Ishita Chimnani is the Builder / Integrator, who develops the second feature branch and the merge conflict, runs Docker commands 13 to 25, adds the phpmyadmin and redis services and

runs the Compose lifecycle commands, builds the Scripted pipeline, creates the Jenkins job that uses the Jenkinsfile from GitHub, handles the Jira smart commit, pull request and second branch, sets up the Nagios remote agent and adds the host in Nagios.

Experiment	Heenal Dhanwani	Ishita Chimnani
EXP 01 Git	Steps 1–13 (setup, first push); 14–19 (menu and ordering branch); 26 (merge); 31, 32 (history, branches); 33 (soft reset); 35 (restore)	Clone; Steps 20–25 (chatbot assistant branch); 27–30 (merge conflict); 34 (revert)
EXP 02 Docker	Installation 1–6; Dockerfile and image build; commands 1–12	Installation 1–6; image build; commands 13–25 and Linux commands
EXP 03 Compose	web, app and database services; down, up -d, config	phpmyadmin and redis services; ps, logs, images; stop, start, restart, down, down -v, config
EXP 04 Jenkins CI	Procedure 1–3 (JDK, jenkins.war, unlock, plugins, admin); create user; Procedure 4 and 5: Declarative pipeline	Procedure 4 and 5: Scripted pipeline
EXP 05 Jenkins + GitHub	Part 1 (verify Jenkins); Part 4 (Jenkinsfile in GitHub); Part 5 (connect Jenkins to GitHub); webhook	Part 2 (first job); Part 3 (run locally); Part 6 (verify build)
EXP 06 Jira	Steps 1–5; 7 (rule: Done); 8 (review and merge); 9, 10 (GitHub Actions); 11 (review and merge SRT-5); 12 (sprint completion)	Step 6 (smart commit); 7 (rule: In Review); 8 (pull request); 11 (second branch, conflict)
EXP 07 Nagios	Steps 1–4 (Nagios server); 7 (verify)	Steps 5–6 (NSClient++ on the remote host); 8–9 (add host, check in web interface)

Table 1.2: Master distribution of experiment steps among the members

2. Requirements Analysis

This chapter records the requirements of the Smart Restaurant Management System: who uses it, what it must do and the qualities it must have. The work itself was planned and tracked as a Scrum project in Jira, which is documented together with the other experiments in Chapter 5 (Section 5.6).

2.1 User Roles

Role	Goal	Main screens
Customer	Browse the menu, place an order from the table and ask questions about dishes or an order	Menu page with cart, Restaurant Assistant chatbot
Restaurant staff (admin)	Track orders, change order status, view sales and manage the menu items	Admin dashboard (/admin)
Operator	Keep the service available and observe its health	/health endpoint, Nagios web interface

Table 2.1: User roles

2.2 Functional Requirements

- The customer can view the menu, filter the dishes by category and see whether a dish is vegetarian or non-vegetarian, with prices in rupees.
- The customer can add dishes to a cart, change the quantities, enter a name and table number and place an order.
- The application calculates the order total on the server from the stored prices and returns an order number.
- The Restaurant Assistant chatbot answers questions about the menu, prices, vegetarian dishes, categories, recommendations, opening hours and the status of an order.
- Staff sign in to the admin dashboard to see orders and sales statistics, change the status of an order (Placed, Preparing, Ready, Served, Cancelled) and add or hide menu items.
- Orders and the menu are stored in an SQLite database that is created automatically at start-up and filled with a sample menu.
- GET /health returns HTTP 200 with the status of the service.

2.3 Non-Functional Requirements

- **Performance:** menu and order requests are small JSON calls served directly from SQLite, so the pages respond immediately on a developer machine.
- **Security and integrity:** prices and totals are read from the database on the server, quantities and inputs are validated, and the admin pages need a password that is set through the ADMIN_PASSWORD environment variable.
- **Portability:** the application starts with python app/app.py on port 5000, in a Docker container, or with docker compose up -d behind nginx.
- **Maintainability:** automated pytest tests run on every push and pull request through GitHub Actions.
- **Observability:** the /health endpoint can be checked by Nagios and by Docker, and the host is monitored through NSClient++.

3. System Design

3.1 Architecture

The Smart Restaurant Management System follows a layered architecture. The presentation layer is a set of HTML templates (index.html for customers and admin.html for staff) with CSS and JavaScript served by Flask. When the Compose stack is used, requests reach nginx on port 8081, which acts as a reverse proxy and passes them to the Flask backend on port 5000. In the backend, an HTTP router exposes GET /, GET /health, GET /api/menu, POST /api/orders, GET /api/orders/<id>, POST /api/chat and the protected /admin pages with their /api/admin endpoints. The order logic validates the cart and calculates the total from the stored prices, and the chatbot engine matches the message against intents and answers from the database. All data is stored in an SQLite database (restaurant.db) with the tables menu, orders and order_items. Jenkins and Docker form the containerisation and CI/CD layer that builds, tests and deploys the backend.

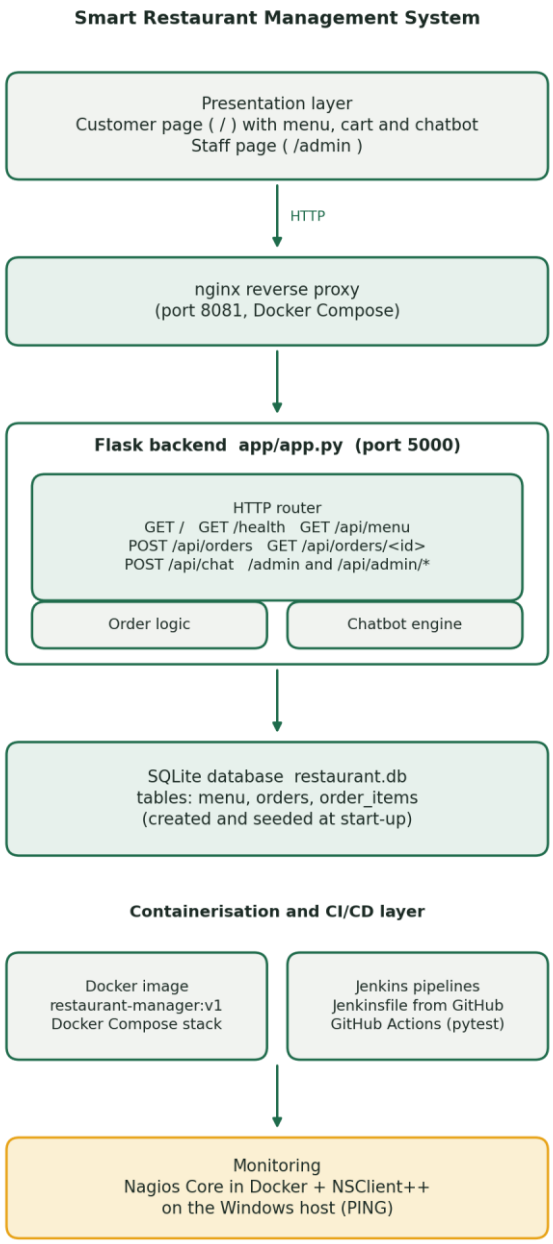


Figure 3.1: System architecture of the Smart Restaurant Management System

3.2 Components

Layer	Component	Responsibility
Presentation	templates/index.html, templates/admin.html, static/style.css, static/app.js	Menu with category filter, cart and order form, floating chatbot window, and the staff dashboard for orders, sales and menu management
Reverse proxy	nginx (port 8081)	Routes incoming traffic to the backend
Backend	Flask app/app.py (port 5000)	HTTP router with GET /, GET /health, /api/menu, /api/orders, /api/chat and the /admin pages

Layer	Component	Responsibility
Order logic	Cart validation and total calculation	Reads prices from the database, checks the quantities (1 to 20) and stores the order with its items
Chatbot	Rule-based intent engine (regular expressions)	Recognises greeting, order status, dish price, vegetarian, category, cheapest item, recommendation, menu and opening-hours questions and answers from the database
Database	SQLite (restaurant.db: menu, orders, order_items)	Created and seeded automatically at start-up; the MySQL and Redis containers of Compose are demonstration services
DevOps	Docker, Docker Compose, Jenkins, GitHub Actions, Nagios	Packaging, orchestration, build, test, deployment and monitoring

Table 3.1: Components of the system

3.3 Ports and Machines

Service	Port	Where it runs
Application (Flask)	5000	Each member's own PC
Jenkins	8080	Heenal Dhanwani's PC (Ishita Chimnani opens it over the network)
Compose nginx (web)	8081	Each member's own PC
Compose phpMyAdmin	8082	Each member's own PC
Nagios web interface	8090	Heenal Dhanwani's PC (8090 is used because Jenkins already uses 8080)
NSClient++ (NRPE)	5666	Ishita Chimnani's PC, the remote Windows host that is monitored

Table 3.2: Ports and machines used in the project

4. Application Implementation

4.1 Menu and Cart

The customer page shows the restaurant name, the opening hours and the menu. Category buttons (All, Starters, Main Course, Breads, Desserts, Beverages) filter the dishes, and a green or red square before each name marks a vegetarian or non-vegetarian dish. Pressing Add puts a dish in the Your order panel, where the quantity can be changed with the plus and minus buttons and the total is updated in rupees. The customer enters a name and a table number and presses Place order; the page then shows the order number and the total.

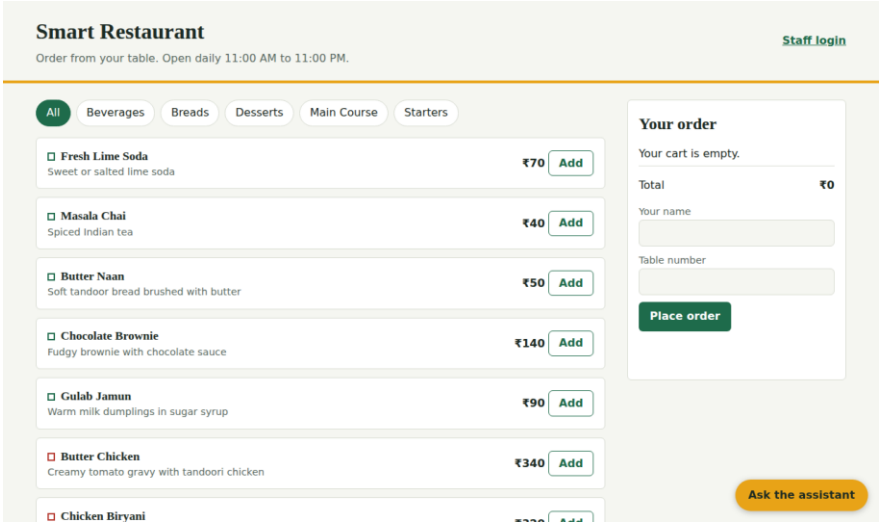


Figure 4.1: Menu page of the Smart Restaurant Management System with category filters

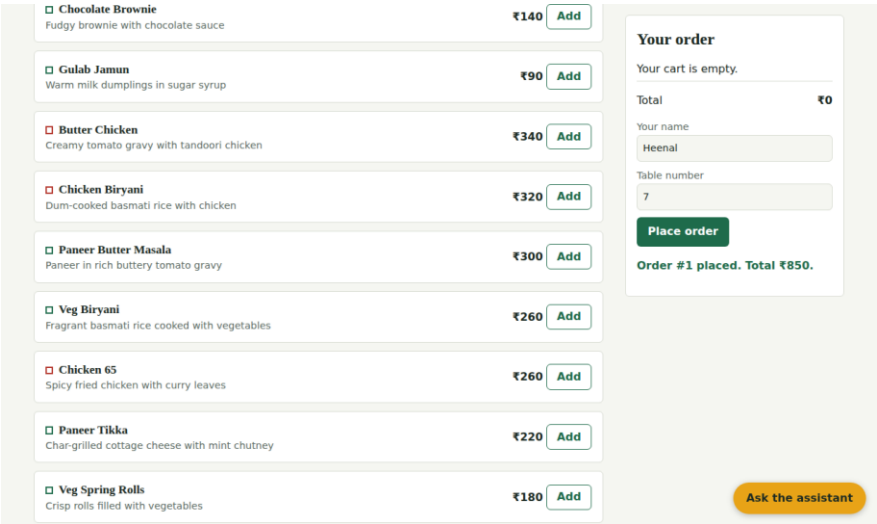


Figure 4.2: Cart with items, name and table number, and the order confirmation

4.2 Chatbot and Admin Dashboard

The Ask the assistant button opens the Restaurant Assistant. The customer can type questions such as show the menu, price of butter chicken, vegetarian dishes or status of order 1, and the chatbot answers from the database. Staff open /admin, sign in with the admin password and see the sales summary and the top-selling dishes, every order with a drop-down to change its status, and the menu list on which items can be added or hidden. A status change made by staff is immediately reported by the chatbot.

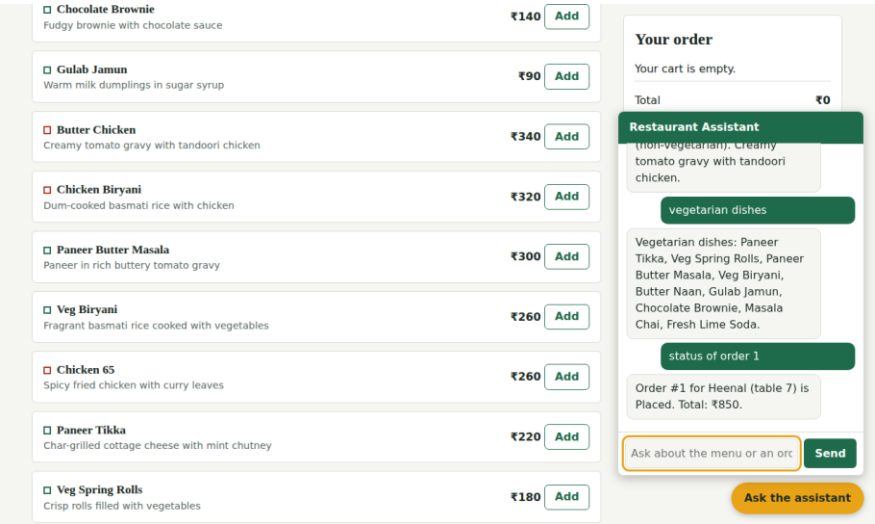


Figure 4.3: Restaurant Assistant chatbot answering menu, price and order-status questions

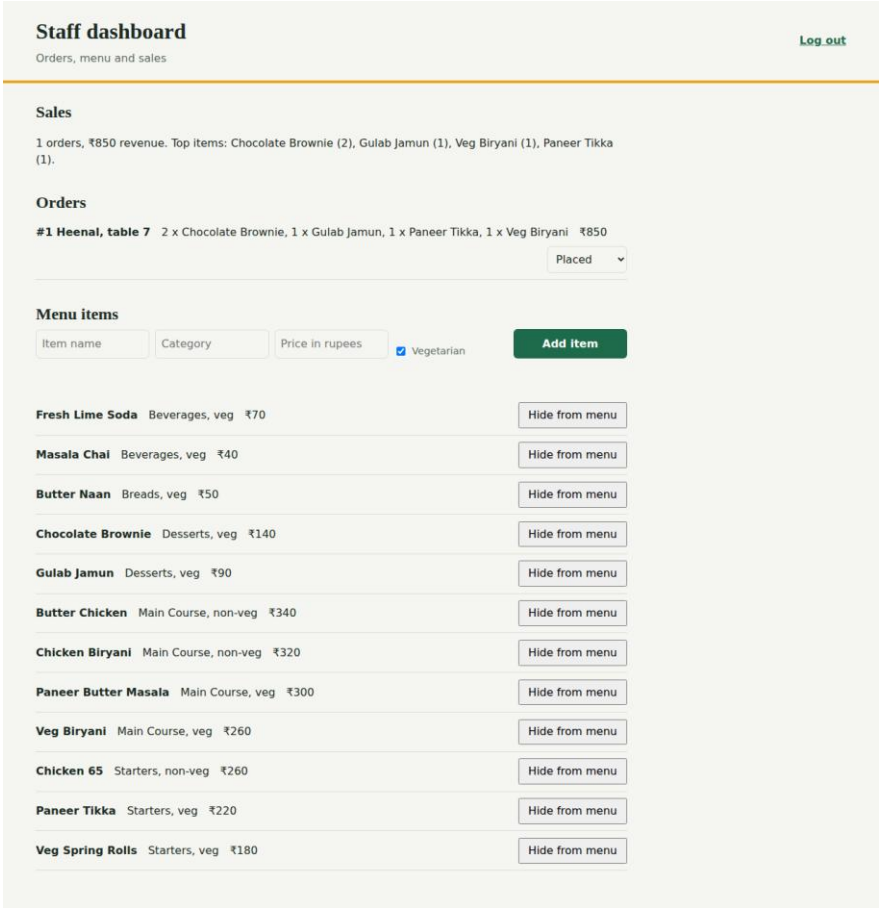


Figure 4.4: Staff dashboard with sales, orders and menu management

4.3 Order and Chatbot Processing

1. The browser loads the menu with GET /api/menu, and the cart is kept in the page until the order is placed.
2. On Place order the browser sends the name, the table number and the cart items to POST /api/orders.

3. The server checks every item and quantity against the menu table, calculates the total from the stored prices, and rejects an empty cart or an unavailable item.
4. The order and its items are saved in the orders and order_items tables and the order number is returned; staff change the status of the order through the admin API.
5. A chat message sent to POST /api/chat is matched against the intents in a fixed order (order status, dish name, vegetarian, category, recommendation, menu, opening hours) and the reply is built from the database.

4.4 Health Endpoint

GET /health returns a small JSON document, {"status": "ok"}, with HTTP 200. It is used to confirm that the application runs locally, in a container and behind nginx, and it is the target of the automated tests and the monitoring checks described in later chapters.

5. DevOps Implementation

This chapter documents Experiments 1 to 7 as applied to the Smart Restaurant Management System. For each experiment it gives the aim, the theory, the procedure with the commands, the output captured by each member, and a conclusion.

5.1 Version Control with Git and GitHub (Experiment 1)

5.1.1 Aim

To perform version control operations using Git (LO2: perform version control operations using Git and apply collaboration workflows). On the Smart Restaurant Management System the project is placed under Git and pushed to GitHub, two feature branches are developed by two members and merged, a merge conflict is created and resolved, and rollback, revert and restore are practised.

5.1.2 Theory

Version control records changes to files so that earlier versions can be recalled and several people can work on one project. Git is distributed: every developer holds the full repository and its history, and shares commits through a remote such as GitHub. Changes move from the working directory to the staging area (git add) and into the repository as a commit; a branch isolates a feature, and merging joins branches with a fast-forward or a merge commit. When two branches change the same lines, Git reports a merge conflict and marks the file with <<<<<<<, ======= and >>>>>>>, which the developer removes after choosing the final content.

Work is shared with a remote (origin) using git clone, push, fetch and pull. Mistakes are undone with git reset --soft HEAD~1 (drop the last commit but keep the changes staged), git revert (a new commit that reverses an earlier one, safe for shared branches) and git restore (discard uncommitted changes).

Command	Purpose
git init / git clone	Create a new repository / copy an existing one
git config	Set user name, email and other settings
git status / git add / git commit	Show changes / stage changes / record a snapshot
git branch / git checkout -b	List branches / create and switch to a branch
git merge	Join another branch into the current one
git push / git pull / git fetch	Upload / download and merge / download commits
git log --oneline --graph --all	Show the commit history with branch visualisation
git reset --soft / git revert / git restore	Roll back a commit / reverse a commit by a new one / discard uncommitted changes

Table 5.1: Git commands used in Experiment 1

5.1.3 Implementation

Steps 1 to 13

```
mkdir smart-restaurant-system
cd smart-restaurant-system
git init
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git config --list
echo "Smart Restaurant Management System Collaboration Project" > project.txt
git status
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/heenaldhanwani/smart-restaurant-system.git
git remote -v
git push -u origin main
```

Steps 14 to 19 and 26

```
git clone https://github.com/heenaldhanwani/smart-restaurant-system.git        # Ishita Chimnani (team step)
cd smart-restaurant-system
git checkout -b feature-menu-ordering          # Heenal Dhanwani, Step 14
echo "Menu and Ordering Module" >> project.txt
git add .
git commit -m "Added menu and ordering module"
git push origin feature-menu-ordering
git checkout main
git merge feature-menu-ordering              # Step 26
git push origin main
```

Steps 20 to 30

```
git checkout -b feature-chatbot-assistant
echo "Chatbot Assistant Module" >> project.txt
git add .
git commit -m "Added chatbot assistant module"
git push origin feature-chatbot-assistant
git checkout main
git pull origin main
git merge feature-chatbot-assistant
# CONFLICT in project.txt: remove the three marker lines and keep both module lines
git add project.txt
git commit -m "Resolved merge conflict"
git push origin main
```

Ishita Chimnani created the chatbot assistant branch from main before Heenal Dhanwani merged, which is what makes both branches change the same part of project.txt. On Windows, the echo command of PowerShell 5.1 saves files as UTF-16, which Git treats as binary; the echo steps were therefore run in Command Prompt or Git Bash.

History, rollback, revert and restore: Steps 31 to 35

```
git pull origin main
git fetch --all
git log --oneline --graph --all        # Heenal Dhanwani, Step 31
git branch -a                          # Heenal Dhanwani, Step 32
git checkout -b practice-reset         # Ishita Chimnani, Step 33
git reset --soft HEAD~1
git checkout -b practice-revert        # Ishita Chimnani, Step 34
git revert <commit-id>
git restore .                          # Heenal Dhanwani, Step 35
```

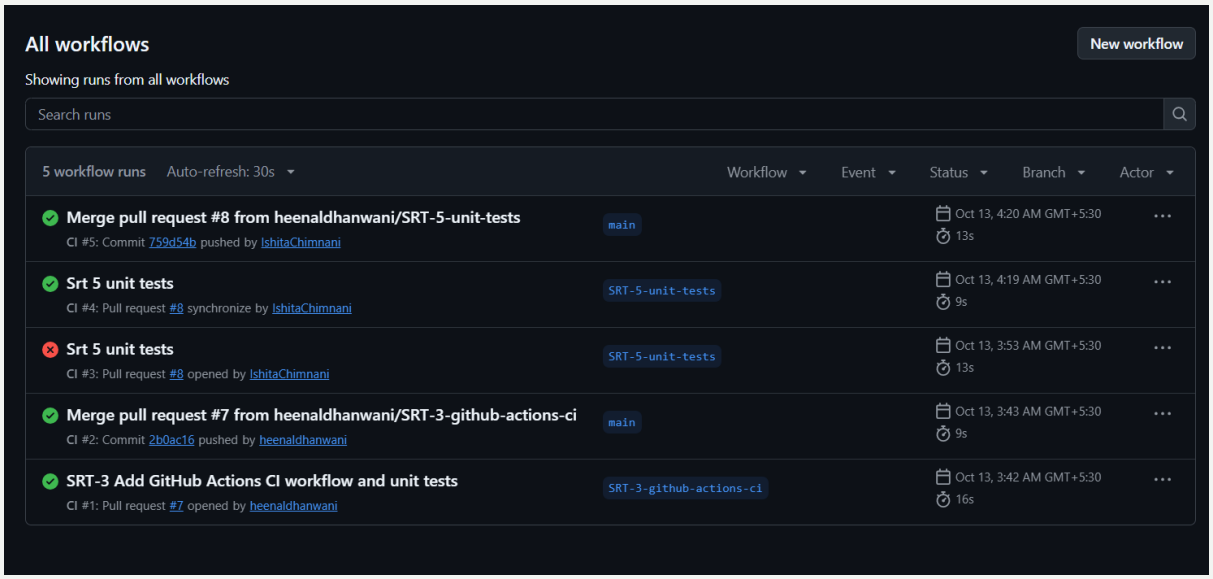
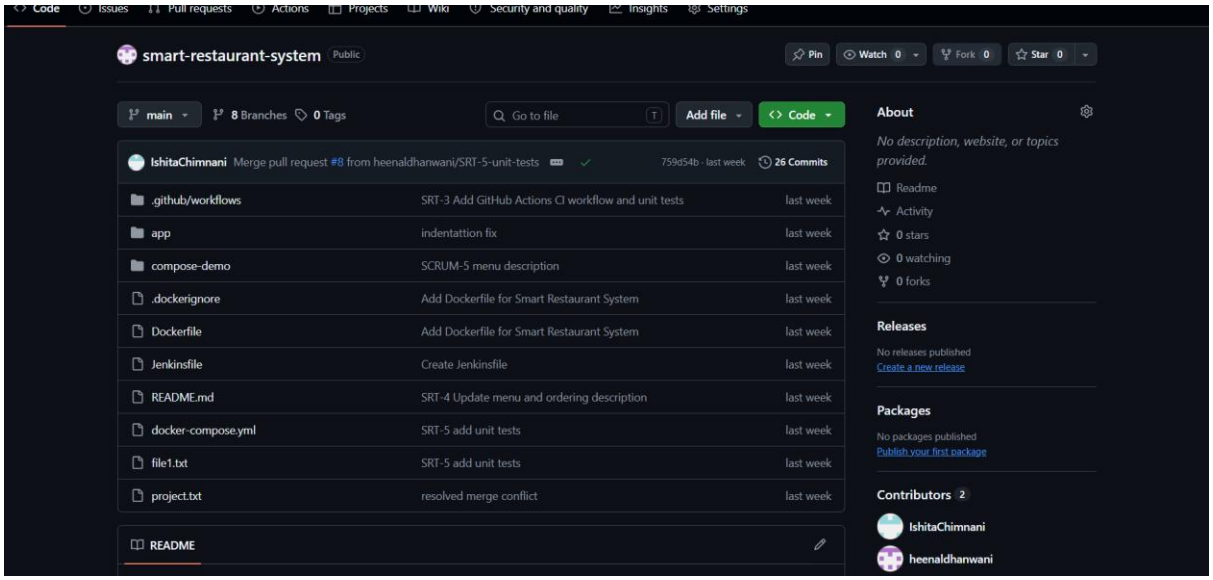


Figure 5.1: Output screenshots of Experiment 1 (Git and GitHub)




```
MINGW64:/c/Users/heena/smart-restaurant-system
heena@HEENAL MINGW64 ~/smart-restaurant-system (main)
$ git log
commit 759d54bbb482067b97bc904d7edb202a573c9ff7 (HEAD -> main, origin/main, origin/HEAD)
Merge: 2b0ac16 a577e9e
Author: IshitaChimnani <ishitakiara@gmail.com>
Date:   Tue Oct 13 04:20:44 2026 +0530

    Merge pull request #8 from heenaldhanwani/SRT-5-unit-tests

    Srt 5 unit tests

commit a577e9ef87c6e1b68b21575d62f3b0e849a71eb7
Author: Ishita <ishitakiara@gmail.com>
Date:   Tue Oct 13 04:18:47 2026 +0530

    indentattion fix

commit 94849e9d093d27391bd2ca9cdb2c6a5736ac646d
Merge: 27d9b00 2b0ac16
Author: Ishita <ishitakiara@gmail.com>
Date:   Tue Oct 13 03:50:43 2026 +0530

    SRT-5v resolved conflict

commit 2b0ac165b4ad317d7116e404cab4d39fc9b05b5a
Merge: 24d85ce 32b4a5b
Author: heenaldhanwani <heenaldhanwani@gmail.com>
Date:   Tue Oct 13 03:43:16 2026 +0530

    Merge pull request #7 from heenaldhanwani/SRT-3-github-actions-ci

    SRT-3 Add GitHub Actions CI workflow and unit tests

commit 27d9b000efdb8340d24d22c146ab5b4231df2b9a
Author: Ishita <ishitakiara@gmail.com>
Date:   Tue Oct 13 03:38:41 2026 +0530

    SRT-5 add unit tests

commit 32b4a5bac0702619fc4e3cb4b16264703d769f54 (origin/SRT-3-github-actions-ci, SRT-3-github-actions-ci)
Author: Heenal <heenaldhanwani@gmail.com>
Date:   Tue Oct 13 03:30:31 2026 +0530

    SRT-3 Add GitHub Actions CI workflow and unit tests

commit 24d85cedf0642dcc2a61da93e0f5927fe5a29485
Merge: 37aede8 3e13295
Author: IshitaChimnani <ishitakiara@gmail.com>
Date:   Tue Oct 13 03:11:38 2026 +0530

    Merge pull request #6 from heenaldhanwani/SRT-4-menu-ordering

    SRT-4 Update menu and ordering description

commit 3e132958c3c5ea29a9430b856407a58c302fb5c0
Author: Ishita <ishitakiara@gmail.com>
Date:   Tue Oct 13 02:58:49 2026 +0530
```

5.1.4 Conclusion

A Git repository was created and connected to GitHub, feature branches were created and merged, a merge conflict was resolved, and rollback, revert, restore and history operations were performed.

5.2 Containerisation with Docker (Experiment 2)

5.2.1 Aim

To containerise the application using Docker **(LO4: containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools).** The Smart Restaurant Management System is packaged as a Docker image and run as a container, and the 25 Docker commands of the lab file are performed, each member running them on their own PC.

5.2.2 Theory

A container packages an application with its libraries and settings and runs it as an isolated process that shares the host kernel, so it starts in seconds and behaves the same on every machine. An image is a read-only, layered blueprint described by a Dockerfile and identified by a name and tag (for example restaurant-manager:v1); a container is a running instance of an image. On Windows, Docker Desktop runs the engine on the WSL 2 backend, which needs hardware virtualisation to be enabled in the BIOS.

5.2.3 Implementation

Installation (all members)

1. Enable Intel VT-x or AMD-V in the BIOS/UEFI settings (check in Task Manager, Performance, CPU, Virtualization: Enabled).
2. Install WSL 2 with wsl --install in PowerShell as Administrator and restart the computer.
3. Download and install Docker Desktop with the default settings (WSL 2 recommended).
4. Launch Docker Desktop and wait until the engine shows as running.
5. Verify the installation with docker --version and docker run hello-world.

Dockerfile, image build and container

Heenal Dhanwani wrote the Dockerfile and .dockerignore, committed them on the branch docker-setup and merged it into main through a pull request. Ishita Chimnani then pulled main and built the image on her own PC.

```
FROM python:3.11-slim
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
EXPOSE 5000
CMD ["python", "app.py"]
```

```
git checkout -b docker-setup
git add Dockerfile .dockerignore
git commit -m "Add Dockerfile for Smart Restaurant Management System"
git push origin docker-setup

docker build -t restaurant-manager:v1 .
docker run -d --name restaurant-app -p 5000:5000 restaurant-manager:v1
```

The containerised application is confirmed by opening http://localhost:5000/health.

Docker commands 1 to 25

No.	Command	Purpose
1	docker --version	Check the Docker version
2	docker search ubuntu --filter stars=500	Search Docker Hub with a filter
3	docker pull ubuntu	Download an image
4	docker run -it ubuntu /bin/bash	Run an interactive container
5	docker ps	List running containers
6	docker run --name mycontainer -it ubuntu bash	Create and start a named container
7	docker stop mycontainer	Stop a container gracefully
8	docker start mycontainer	Restart a stopped container
9	docker inspect mycontainer	Show detailed information about an object
10	docker ps -a	Show running and stopped containers
11	docker stats	Real-time resource usage
12	docker rm -f mycontainer	Remove a container (forced)
13	docker run -d --name web nginx	Run nginx in the background
14	docker pull alpine	Download the Alpine Linux image
15	docker search python	Search for an image
16	docker image prune	Remove dangling images
17	docker container ls	List running containers
18	docker info	System-wide information
19	docker history <image>	Layers of an image
20	docker system df	Disk space used by Docker

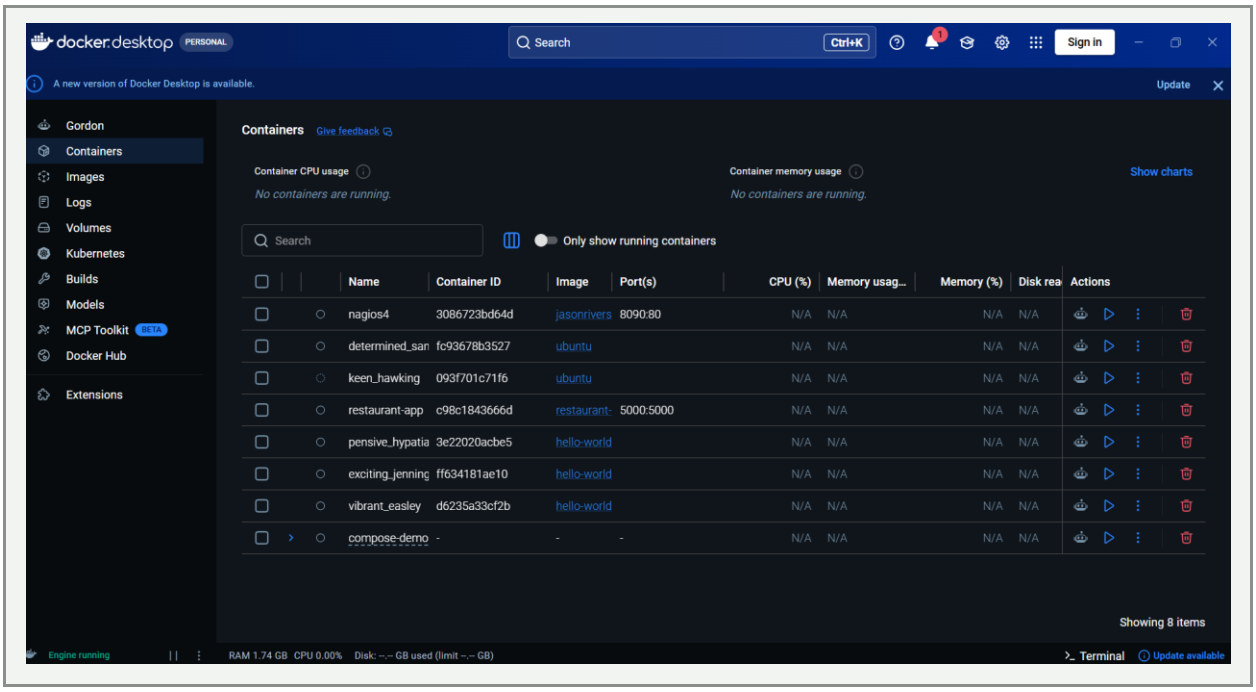
No.	Command	Purpose
21	docker network inspect bridge	Details of a network
22	docker volume create restaurant-data	Create persistent storage
23	docker network create restaurant-net	Create a network
24	docker system prune	Remove unused objects
25	docker rename restaurant-app restaurant-manager-app	Rename a container

Table 5.2: The 25 Docker commands and their purpose

Commands 1 to 12 (Heenal Dhanwani)

Image build and commands 13 to 17 (Ishita Chimnani)

Installation and commands 18 to 25 (Ishita Chimnani)



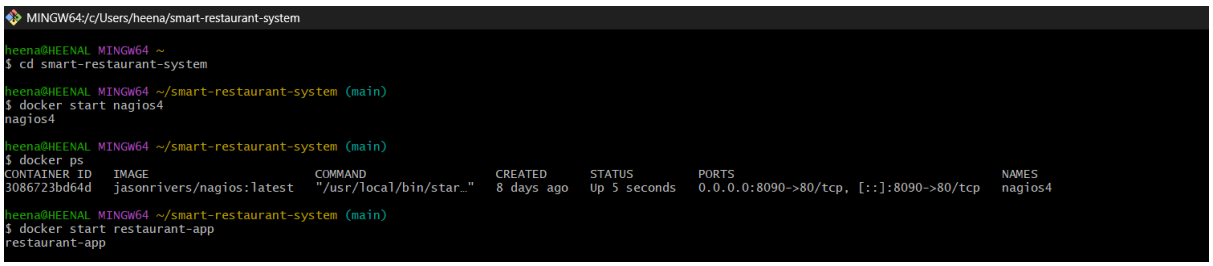


Figure 5.2: Output screenshots of Experiment 2 (Docker)

5.2.4 Conclusion

Docker packages the application with its dependencies into a container, giving a lightweight and consistent environment. The difference between an image (a read-only blueprint) and a container (a running instance) was demonstrated with the restaurant manager image and container.

5.3 Multi-service Deployment with Docker Compose (Experiment 3)

5.3.1 Aim

To deploy multi-service applications using Docker Compose. The application and its supporting services (nginx web server, MySQL database, Redis cache and phpMyAdmin) are defined in one docker-compose.yml file and started with one command.

5.3.2 Theory

Docker Compose describes a multi-container application in one YAML file and manages it as a unit. Each entry under services: defines a container with its image or build context, ports, environment variables, volumes and dependencies. Compose creates a project network on which services reach each other by service name (for example phpMyAdmin connects to the host database and nginx forwards to http://app:5000), and a volume keeps database data beyond the life of a container. Ports are mapped host:container, so a port conflict is solved by changing the left-hand number.

Compose command	Purpose
docker compose config	Validate and display the configuration
docker compose up -d	Create and start all services in the background
docker compose ps / logs / images	Show container status / logs / images used
docker compose stop / start / restart	Stop, start or restart containers without removing them
docker compose down	Stop and remove containers and networks
docker compose down -v	Also remove the volumes

Table 5.3: Docker Compose commands

5.3.3 Implementation

The two members built one docker-compose.yml together. Each added their services on their own branch (compose-a, compose-b) and merged through a pull request in the order A, B. Containers left over from Experiment 2 were stopped first so that the ports were free.

Heenal Dhanwani: services web, app and database

```
docker --version
mkdir compose-demo
cd compose-demo
```

```
services:
```

```
  web:
    image: nginx
    ports:
      - "8081:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
  app:
    build:
      context: ..
      dockerfile: Dockerfile
    environment:
      REDIS_HOST: redis
  database:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root123
```

```
server {
    listen 80;
    location / {
        proxy_pass http://app:5000;
    }
}
```

```
docker compose down
docker compose up -d
docker compose config
docker compose up -d
```

Ishita Chimnani: services phpmyadmin and redis, and lifecycle commands

```
phpmyadmin:
  image: phpmyadmin
  ports:
    - "8082:80"
  environment:
    PMA_HOST: database
redis:
  image: redis
```

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs
docker compose images
```

Lifecycle commands (Ishita Chimnani)

```
docker compose config
docker compose up -d
docker compose ps
docker compose stop
docker compose start
docker compose restart
docker compose down
docker compose up -d
docker compose down -v
docker compose config
```

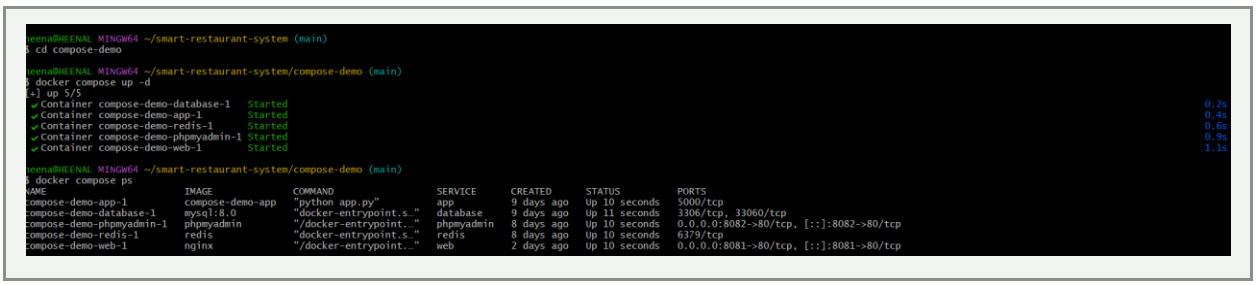


Figure 5.3: Output screenshots of Experiment 3 (Docker Compose)

5.3.4 Conclusion

Docker Compose created and managed a multi-container application from one YAML file. Services, images, environment variables and port mapping were used, and the essential Compose commands were performed, including resolving port conflicts.

5.4 Continuous Integration with Jenkins (Experiment 4)

5.4.1 Aim

To understand Continuous Integration, install Jenkins, and build CI/CD pipelines using Declarative and Scripted syntax (LO3). A Jenkins server is installed and two pipeline jobs, one Declarative and one Scripted, run Build, Test and Deploy stages for the Smart Restaurant Management System.

5.4.2 Theory

Continuous Integration (CI) merges every developer's changes into the shared branch frequently and verifies each merge with an automatic build and tests; Continuous Delivery / Deployment (CD) extends this by preparing and releasing every tested build automatically. Jenkins is an open-source automation server written in Java, started here with java -jar jenkins.war on port 8080 and extended by plugins. A Pipeline job expresses the build as code, divided into stages (Build, Test and Deploy) that contain steps such as echo.

Aspect	Declarative Pipeline	Scripted Pipeline
Outer structure	pipeline { agent, stages, stage, steps }	node { stage(...) { ... } }
Syntax	Structured and predefined, easy to read and validate	Free-form Groovy, more flexible
Error handling	Built-in post sections	Written by hand with try / catch
Typical use	Most projects and the Jenkinsfile stored in source control	Complex logic that needs full programming control

Table 5.4: Declarative and Scripted pipelines compared

5.4.3 Implementation

Procedure steps 1 to 3

```
java --version
cd C:\jenkins
java -jar jenkins.war
```

Jenkins was opened at http://localhost:8080 and unlocked with the initial administrator password (printed in the PowerShell window and stored in C:\Users\<you>\.jenkins\secrets\initialAdminPassword). Install suggested plugins was selected, the administrator account was created and the default Jenkins URL was kept. To let Ishita Chimnani open Jenkins from her own PC, the firewall rule below was added (PowerShell as Administrator) and a user was created for her under Manage Jenkins, Users, Create User.

```
ipconfig
New-NetFirewallRule -DisplayName "Jenkins 8080" -Direction Inbound -LocalPort
8080 -Protocol TCP -Action Allow
```

Declarative Pipeline (Heenal Dhanwani, job RestaurantMgmt-Declarative)

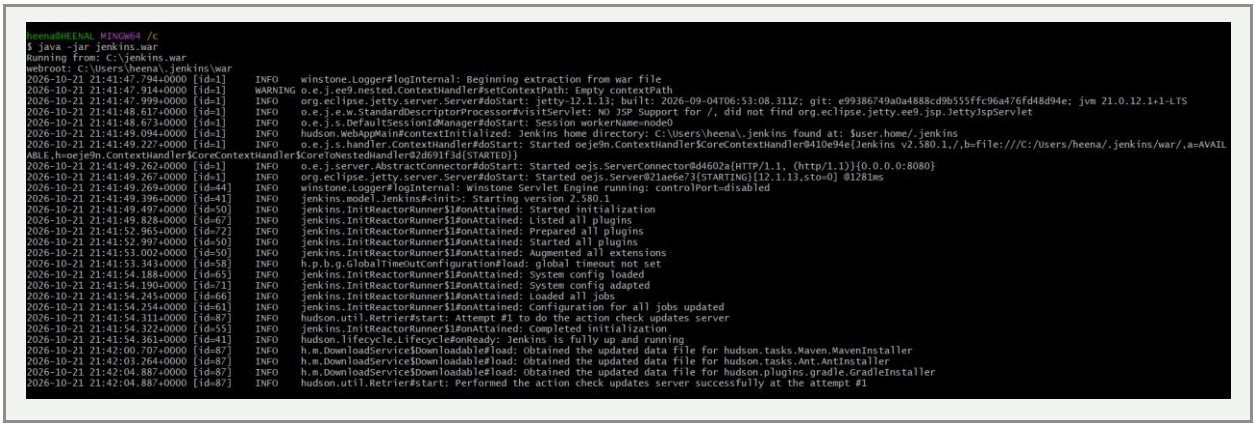
```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the Smart Restaurant Management System...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing the Smart Restaurant Management System...'
            }
        }
        stage('Deploy') {
            steps {
```

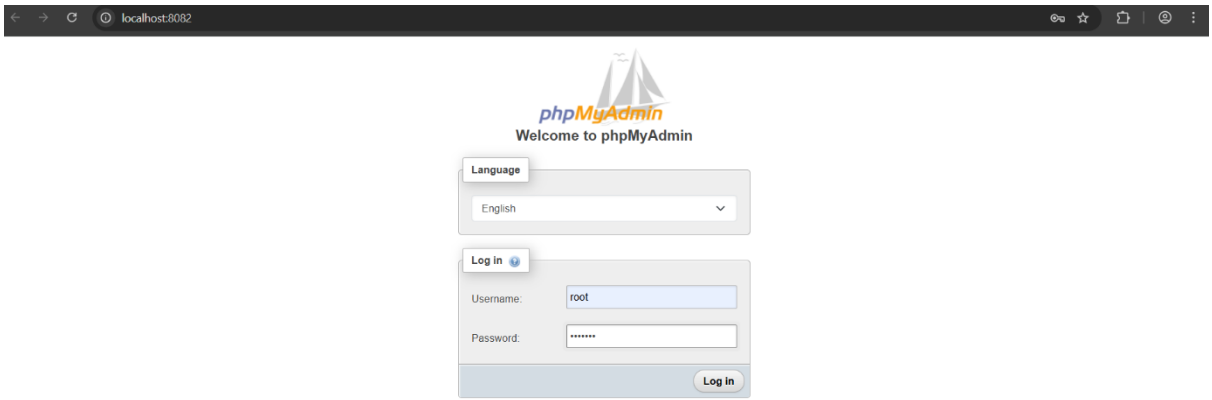
```
            echo 'Deploying the Smart Restaurant Management System...'
        }
    }
  }
}
```

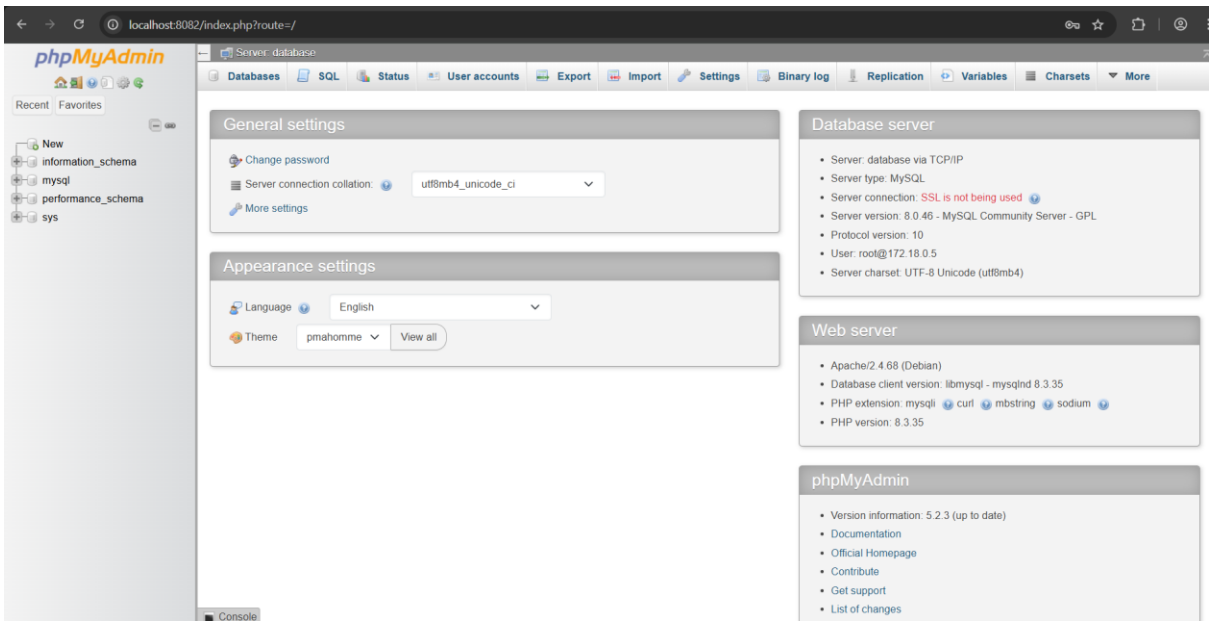
Scripted Pipeline (Ishita Chimnani, job RestaurantMgmt-Scripted)

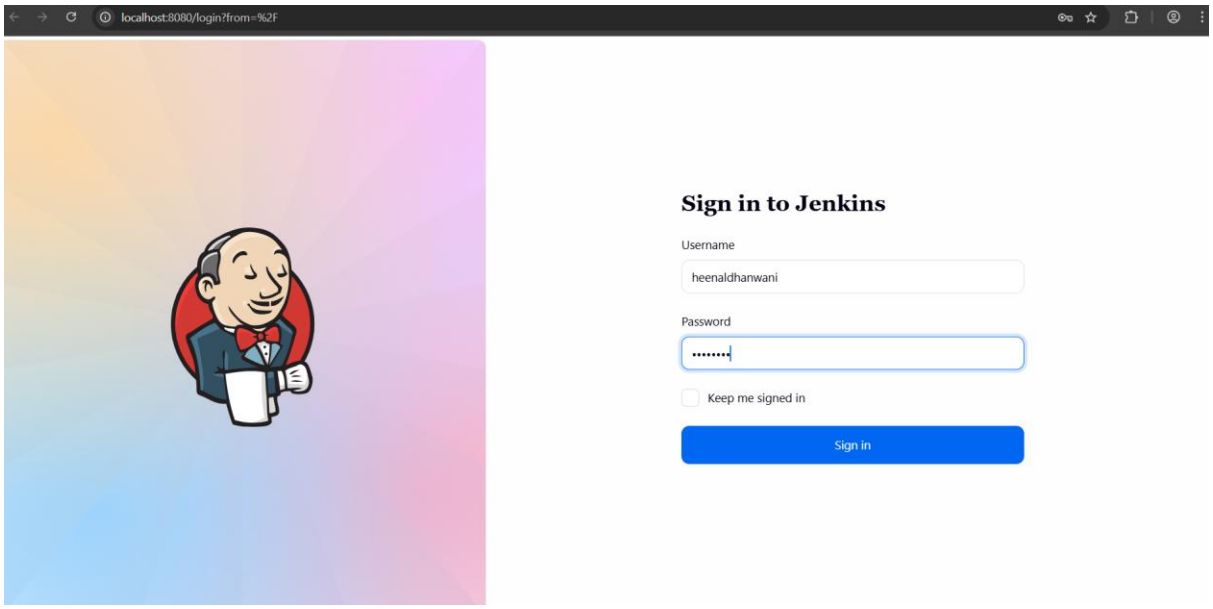
```
node {
   stage('Build') {
      echo 'Building the Smart Restaurant Management System...'
   }
   stage('Test') {
      echo 'Testing the Smart Restaurant Management System...'
   }
   stage('Deploy') {
      echo 'Deploying the Smart Restaurant Management System...'
   }
}
```

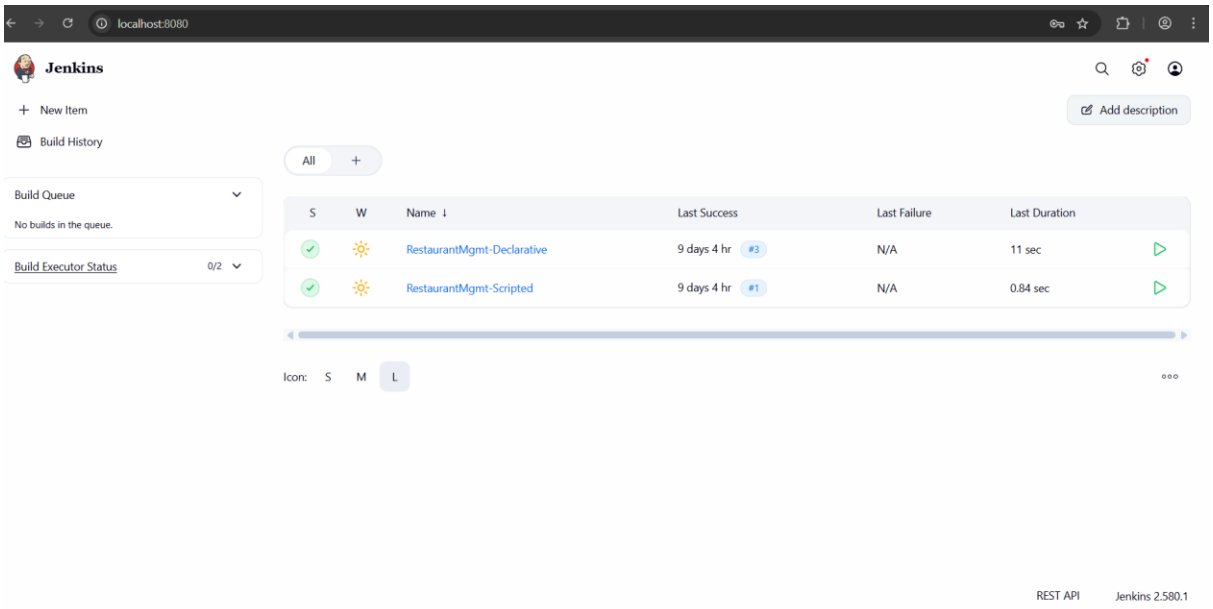
In each case the job was created with New Item, Pipeline, the script was pasted in the Pipeline section and saved, and Build Now was clicked. Build #1 was opened and its Console Output checked for the status and the line Finished: SUCCESS.






localhost:8082/index.php?route=/
phpMyAdmin
Recent
Favorites
New
information_schema
mysql
performance_schema
sys
Server: database
Databases
SQL
Status
User accounts
Export
Import
Settings
Binary log
Replication
Variables
Charsets
More
General settings
Change password
Server connection collation:
utf8mb4_unicode_ci
More settings
Appearance settings
Language
English
Theme
pmahomme
View all
Database server
Server: database via TCP/IP
Server type: MySQL
Server connection: SSL is not being used
Server version: 8.0.46 - MySQL Community Server - GPL
Protocol version: 10
User: root@172.18.0.5
Server charset: UTF-8 Unicode (utf8mb4)
Web server
Apache/2.4.68 (Debian)
Database client version: libmysql - mysqlnd 8.3.35
PHP extension: mysqli curl mbstring sodium
PHP version: 8.3.35
phpMyAdmin
Version information: 5.2.3 (up to date)
Documentation
Official Homepage
Contribute
Get support
List of changes
Console


localhost:8080/login?from=%2F
Sign in to Jenkins
Username
heenaldhanwani
Password
Keep me signed in
Sign in


localhost:8080
Jenkins
New Item
Build History
Build Queue
No builds in the queue.
Build Executor Status
0/2
All
Add description
S
W
Name
Last Success
Last Failure
Last Duration
RestaurantMgmt-Declarative
9 days 4 hr #3
N/A
11 sec
RestaurantMgmt-Scripted
9 days 4 hr #1
N/A
0.84 sec
Icon: S M L
REST API
Jenkins 2.580.1

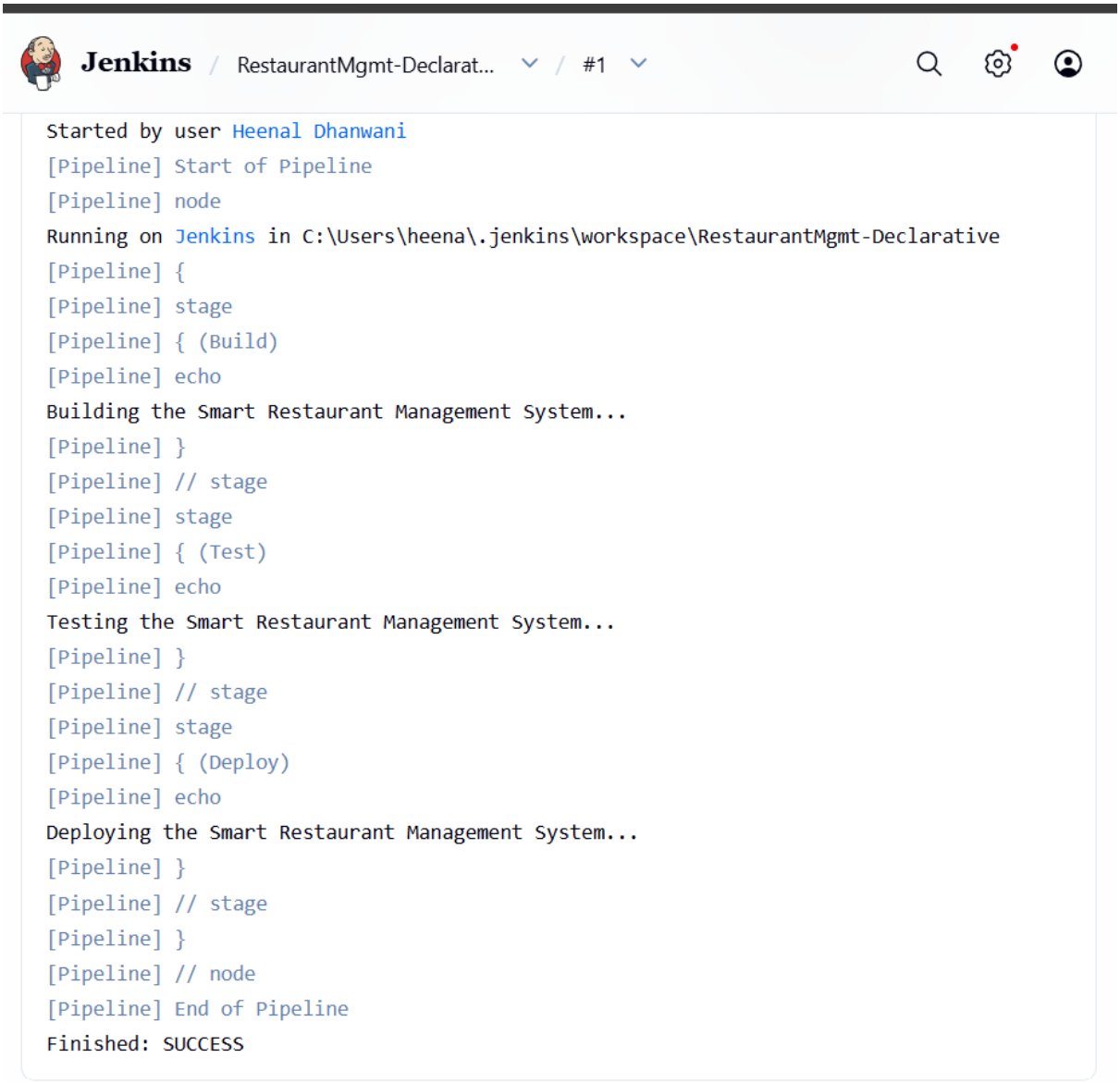


Figure 5.4: Output screenshots of Experiment 4 (Jenkins Declarative and Scripted pipelines)

5.4.4 Conclusion

Jenkins was installed and configured, and both Declarative and Scripted Pipelines were created and executed, showing how Pipeline as Code automates build, test and deploy.

5.5 Automated Deployment with GitHub and Jenkins (Experiment 5)

5.5.1 Aim

To automate application deployment using GitHub and Jenkins (LO3). A Jenkinsfile with Build, Test and Deploy stages is stored in the GitHub repository, Jenkins fetches it with Pipeline script from SCM, and a push to GitHub triggers a build.

5.5.2 Theory

Keeping the pipeline in a Jenkinsfile in the same repository as the code (Pipeline as Code) versions, reviews and rolls it back together with the application; with Pipeline script from SCM, Jenkins clones the repository, checks out */main and reads the Jenkinsfile. To avoid polling,

GitHub sends a webhook (here /github-webhook/) when code is pushed and the option GitHub hook trigger for GITScm polling starts the job; because Jenkins runs on a private PC, a public tunnel (ngrok) exposes port 8080. Build prepares the application, Test runs the automated tests and Deploy places the tested application in its target environment.

5.5.3 Implementation

Heenal Dhanwani: Part 1 and Part 4

Java was checked with java --version, and Jenkins, already installed in Experiment 4, was confirmed to be running at http://localhost:8080. The Jenkinsfile was then added to the repository smart-restaurant-system, in the root, with the commit message Add Jenkins pipeline for build, test, and deploy.

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing the application...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

Ishita Chimnani: Part 2 and Part 3; Heenal Dhanwani: webhook

The job RestaurantMgmt-Demo was created with New Item, Pipeline, the Declarative script was entered directly, and Build Now was clicked. The Console Output of build #1 shows the stages Build, Test and Deploy and ends with Finished: SUCCESS. For the webhook, ngrok was started on Heenal Dhanwani's PC, because only the repository owner can add webhooks, and a webhook was added in the GitHub repository settings.

```
ngrok http 8080

# GitHub: Settings, Webhooks, Add webhook
#   Payload URL : https://<ngrok-id>.ngrok-free.app/github-webhook/
#   Content type: application/json
#   Event       : Just the push event
```

Heenal Dhanwani: Part 5; Ishita Chimnani: Part 6

- Under Triggers, tick GitHub hook trigger for GITScm polling.
- Under Pipeline, change Definition from Pipeline script to Pipeline script from SCM.
- Set SCM to Git, enter the repository URL https://github.com/heenaldhanwani/smart-restaurant-system.git, set Branch Specifier to */main and keep Script Path as Jenkinsfile.
- Save, click Build Now (or push a commit), and open the Console Output of build #2.

The console output of build #2 shows Jenkins obtaining the Jenkinsfile from GitHub, cloning it into the workspace, checking out the latest commit of main, running the Build, Test and Deploy stages and ending with Finished: SUCCESS.

Jenkins – Github Connectivity

Automatic trigger test: a change pushed to GitHub starts a Jenkins build

Jenkins / RestaurantMgmt-Declarat... / #1

```
Started by user Heenal Dhanwani
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\Users\heena\.jenkins\workspace\RestaurantMgmt-Declarative
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the Smart Restaurant Management System...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the Smart Restaurant Management System...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the Smart Restaurant Management System...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Files

main

Go to file T

- .github
- app
- compose-demo
- .dockerignore
- Dockerfile
- Jenkinsfile
- README.md
- docker-compose.yml
- file1.txt
- project.txt

smart-restaurant-system / Jenkinsfile

heenaldhanwani Create Jenkinsfile

Code | Blame | 20 lines (20 loc) · 271 Bytes

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing the application...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

Figure 5.5: Output screenshots of Experiment 5 (Jenkins with GitHub)

5.5.4 Conclusion

Jenkins was integrated with GitHub by configuring Pipeline script from SCM and enabling the GitHub hook trigger. Jenkins fetched the Jenkinsfile, checked out the latest commit and ran the Build, Test and Deploy stages to SUCCESS.

5.6 Agile Project Lifecycle with Jira and DevOps Integration (Experiment 6)

5.6.1 Aim

To study and implement the Agile Software Project lifecycle using Jira with DevOps Pipeline integration (LO1: apply Agile methodologies and use Jira for sprint planning, task tracking and integrating with the DevOps workflow). A Jira Scrum project holds the work items of the Smart Restaurant Management System, the GitHub repository is linked to Jira with GitHub for Jira, commits and pull requests carry the Jira issue key, Jira automation moves issues across the board, and GitHub Actions runs the tests.

5.6.2 Theory

Agile delivers working software in small increments, welcomes changing requirements and relies on frequent feedback. Scrum, the most widely used Agile framework, works in fixed-length sprints taken from an ordered Product Backlog; its roles are the Product Owner, the Scrum Master and the Developers, and its events are Sprint Planning, the Daily Scrum, the Sprint Review and the Sprint Retrospective. Requirements are written as user stories (As a <role>, I want <capability>, so that <benefit>) and estimated in story points; technical work is a task, a defect is a bug, and related items are grouped under an epic.

Jira is the Atlassian tool used to plan and track this work: a Scrum project provides a backlog, sprints and a board (To Do, In Progress, In Review, Done), and every work item has a key such as SRT-4. The GitHub for Jira app links a GitHub account to Jira, so that a branch name, commit message or pull request title containing an issue key appears in the Development panel of that issue (a smart commit). Jira Automation rules react to events such as Pull request created or Pull request merged by moving the issue to In Review or Done. GitHub Actions is the continuous integration service of GitHub; a workflow in .github/workflows runs the tests on every push and pull request.

5.6.3 Implementation

The work items of the lab file (SRT-3, SRT-4 and SRT-5) are replaced here by the keys that Jira generated for this project, which start with SRT: SRT-3 (CI pipeline), SRT-4 (menu browsing and order placement) and SRT-5 (unit tests), all under the epic SRT-2. The project backlog is shown in the table below. The echo commands that append to requirements.txt were run in Command Prompt or Git Bash (see Section 5.1.3).

Key	Summary
SRT-2	Epic: Restaurant System Core
SRT-3	Task: Set up CI pipeline with GitHub Actions (Heenal Dhanwani)

Key	Summary
SRT-4	Story: Menu browsing and order placement feature (Ishita Chimnani)
SRT-5	Story: Unit tests for the restaurant API and chatbot (Ishita Chimnani)

Table 5.5: Product backlog of project SRT

Heenal Dhanwani: Steps 1 to 5 (space, backlog, sprint, repository, integration)

Step 1: Jira space. A free Atlassian account was created and a Jira Cloud site (smart-restaurant-system.atlassian.net) was set up with the Software Development (Scrum) template. The team space was named Smart Restaurant Team, with the project key SRT. The board uses the standard To Do, In Progress, In Review and Done workflow, and Ishita Chimnani was invited with the Member role.

Step 2: backlog and sprint. An epic named Restaurant System Core was created with three work items under it (the task SRT-3 and the stories SRT-4 and SRT-5), each assigned to a member and given story points. SRT Sprint 1 (13 October to 27 October 2026) was created, the three items were added to it, and the sprint was started. The items then appeared on the Scrum board under To Do. The board is used throughout the experiment: an item moves to In Review when a pull request is opened and to Done when it is merged, and the sprint is completed at the end with the Complete sprint button.

Step 3: local project. The Python application was scaffolded and run locally to confirm it works, and a .gitignore file (.venv/, __pycache__/, *.pyc, *.db) was committed.

```
cd smart-restaurant-system
python -m venv .venv
.venv\Scripts\activate
pip install -r app\requirements.txt
python app\app.py
# open http://localhost:5000/health, then press Ctrl+C

git checkout main
git add .gitignore
git commit -m "Add .gitignore"
git push origin main
```

Step 4: repository and collaborators. The public repository smart-restaurant-system from Experiment 1 was confirmed, and Ishita Chimnani was confirmed under Settings, Collaborators.

Step 5: GitHub–Jira integration. The GitHub for Jira app was installed from Apps in Jira, the GitHub account was connected, and the app was given access to the repository smart-restaurant-system. Jira confirmed that the account was connected, so that development activity is reflected on Jira issues.

Ishita Chimnani: Steps 6 to 8 and 11 (smart commit, rule 1, pull request, second branch and conflict)

Step 6: smart commit. A branch whose name contains the issue key was created from main, a change was committed with a message that begins with the issue key, and the branch was pushed. The Development panel of the issue then showed the linked commit.

```
git checkout main
git pull origin main
git checkout -b SRT-4-menu-ordering
echo "Menu and ordering: customers browse the menu, add items to the cart and place orders" >> README.md
git add .
git commit -m "SRT-4 Add menu and ordering feature description"
git push origin SRT-4-menu-ordering
```

Step 7: automation rule 1. In Space settings, Automation, a flow was created with the trigger Pull request created and the action Transition work item to In Review, named PR created - In Review and turned on. **Step 8: pull request.** On GitHub the pull request was opened with the title SRT-4 Menu browsing and order placement, and the issue moved to In Review by itself.

Heenal Dhanwani: Steps 7 and 8 (rule 2, review and merge)

Step 7: automation rule 2. A second flow was created with the trigger Pull request merged and the action Transition work item to Done, named PR merged - Done. **Step 8: review and merge.** The pull request was opened, Files changed was reviewed and the pull request was merged; the issue moved to Done automatically.

Step 11, part 1: second branch. To create a realistic merge conflict, Ishita Chimnani created a second branch (SRT-5-unit-tests) from main before the CI branch was merged, and appended a different line to the last line of app/requirements.txt. **Part 2:** after the CI branch was merged into main, main was merged into the second branch. Both branches had added a last line to the same file, so Git reported a merge conflict. The three marker lines were removed, all three lines (flask, pytest and pytest-cov) were kept, and the result was committed and pushed. The conflict message then disappeared from the pull request, and Heenal Dhanwani reviewed and merged it.

```
git checkout main
git pull origin main
git checkout -b SRT-5-unit-tests
echo pytest-cov >> app/requirements.txt
git add .
git commit -m "SRT-5 Add unit tests for restaurant system"
git push origin SRT-5-unit-tests

git fetch origin
git merge origin/main
# edit app/requirements.txt: remove <<<<<<<, ======= and >>>>>>>, keep both lines
git add app/requirements.txt
git commit -m "SRT-5 Resolve merge conflict in requirements.txt"
```

```
git push origin SRT-5-unit-tests
```

Heenal Dhanwani: Steps 9, 10, 11 (review) and 12 (GitHub Actions and completion)

Step 9: GitHub Actions. A branch was created for the CI task, pytest was added to app/requirements.txt, unit tests were written in app/tests, and a workflow file .github/workflows/ci.yml was added. The workflow runs on a push or pull request to main, sets up Python 3.11, installs the requirements and runs pytest.

```
git checkout main
git pull origin main
git checkout -b SRT-3-github-actions-ci
echo pytest >> app/requirements.txt
pip install pytest
python -m pytest app/tests
```

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r app/requirements.txt
      - run: python -m pytest app/tests
```

```
git add .
git commit -m "SRT-3 Add GitHub Actions CI workflow and unit tests"
git push origin SRT-3-github-actions-ci
```

Step 10: verification. A pull request was opened for the CI branch, the checks at the bottom of the pull request showed that all checks had passed, and the pull request was merged into main. **Step 12: final state.** Main was pulled and the history was shown with git log --oneline --graph --all, GitHub Insights, Contributors was opened to show each member's contributions, all sprint items were in Done on the board, and Complete sprint was clicked to finish the sprint.

Definition of Done

A work item is moved to Done only when all of the following hold:

1. The branch name or the commit message contains the Jira issue key, so the work is linked to the ticket.

2. The change reached main through a pull request, and the GitHub Actions check on that pull request is green.
3. The pull request was reviewed and approved by a member other than its author.
4. The merge of the pull request moved the issue to Done through the Jira automation rule.

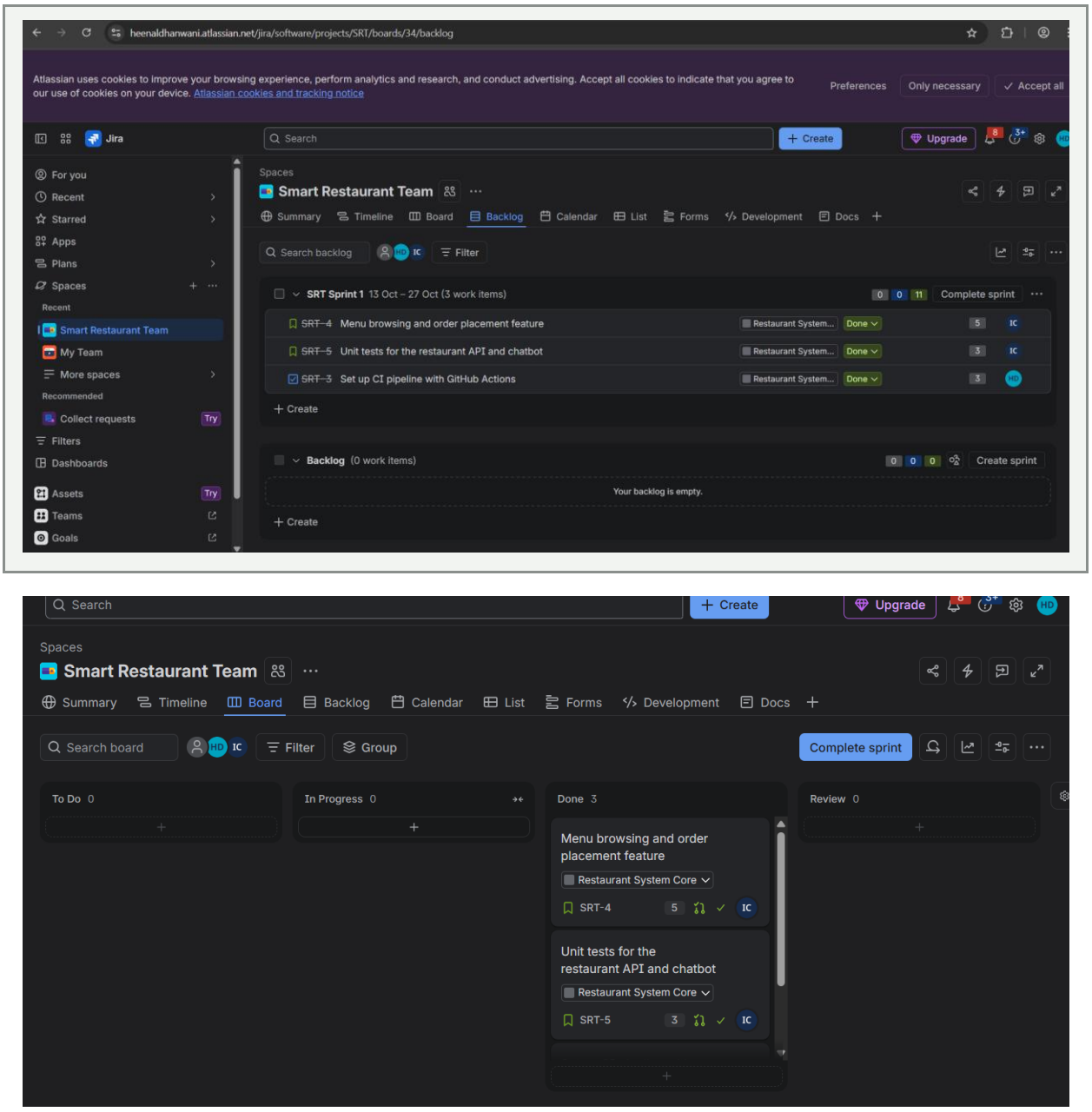


Figure 5.6: Output screenshots of Experiment 6 (Jira, GitHub and GitHub Actions)

5.6.4 Conclusion

Integrating Jira with GitHub established an automated Agile development lifecycle for the Smart Restaurant Management System. The backlog was organised into a sprint, issue keys linked code to tickets, and GitHub Actions and Jira Automation removed manual effort by testing code and moving tickets on pull requests and merges.

5.7 Monitoring with Nagios (Experiment 7)

5.7.1 Aim

To implement monitoring using Prometheus / Grafana / Nagios; Nagios is used **(LO5).** The Nagios Core server runs in a Docker container on Heenal Dhanwani's PC and monitors a Windows host (Ishita Chimnani's PC) through NSClient++.

5.7.2 Theory

Monitoring continuously checks that systems and services are healthy, so that failures are noticed before users complain, and closes the DevOps feedback loop. Nagios Core is configured with objects: a host is a machine to monitor, a service is a check on a host (PING, HTTP, disk space), and every check returns OK, WARNING, CRITICAL or UNKNOWN (UP or DOWN for hosts). Remote hosts are checked by network tests or through an agent: NRPE lets the server ask an agent to run a check, and NSClient++ is the Windows agent with an NRPE server that accepts requests only from its allowed hosts and needs port 5666 open in the firewall.

Nagios Core does not install natively on Windows, so it is run in a container with Docker Desktop (WSL 2 backend), and Windows hosts are monitored with NSClient++. Port 8090 is used for the Nagios web interface because Jenkins already uses 8080.

5.7.3 Implementation

Heenal Dhanwani: Steps 1 to 4 and 7

```
docker --version
wsl --install
wsl --set-default-version 2

docker pull jasonrivers/nagios:latest
docker run --name nagios4 -d -p 8090:80 jasonrivers/nagios:latest
docker ps

docker exec nagios4 htpasswd -bc /opt/nagios/etc/htpasswd.users nagiosadmin
<password>

# Web interface: http://localhost:8090/nagios/  (user nagiosadmin)

docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 172.20.10.2 -n        # Step 7
```

Ishita Chimnani: Steps 5 and 6

```
Invoke-WebRequest -Uri
"https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -
OutFile "NSCP.msi"
Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL
MODULES=CheckExternalScripts,NRPEServer /qn' -Wait
```

```
notepad "C:\Program Files\NSClient++\nsclient.ini"
# [/settings/default]
# allowed hosts = 127.0.0.1,172.20.10.9
# [/settings/NRPE/server]
# use ssl = false
# insecure = true
# verify mode = none
# [/modules]  NRPEServer = enabled, CheckSystem = enabled
Restart-Service nscp

New-NetFirewallRule -DisplayName "NRPE 5666" -Direction Inbound -LocalPort 5666
-Protocol TCP -Action Allow
```

Ishita Chimnani: Steps 8 and 9

```
# vi is not installed in the Nagios image, so the file was created on the PC and copied
into the container
docker cp RestaurantMgmtHost001.cfg
nagios4:/opt/nagios/etc/servers/RestaurantMgmtHost001.cfg
docker exec nagios4 sh -c "echo 'cfg_dir=/opt/nagios/etc/servers' >>
/opt/nagios/etc/nagios.cfg"

define host {
    use                 generic-host
    host_name           RestaurantMgmt_Host_001
    alias               Restaurant Management Host 001
    address             172.20.10.2
    check_command       check-host-alive
    max_check_attempts  2
    check_interval      2
    retry_interval      2
    check_period        24x7
    contact_groups      admins
    notification_period 24x7
    register            1
}

define service {
    host_name           RestaurantMgmt_Host_001
    service_description PING
    check_command       check_ping!100.0,20%!500.0,60%
    max_check_attempts  2
    check_interval      2
    retry_interval      2
    check_period        24x7
    contact_groups      admins
    notification_period 24x7
    register            1
}
```

```
docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg
docker restart nagios4
```

The configuration check should end with Total Errors: 0. After the restart, the host appears under Current Status, Hosts and the PING service under Services within about two minutes.

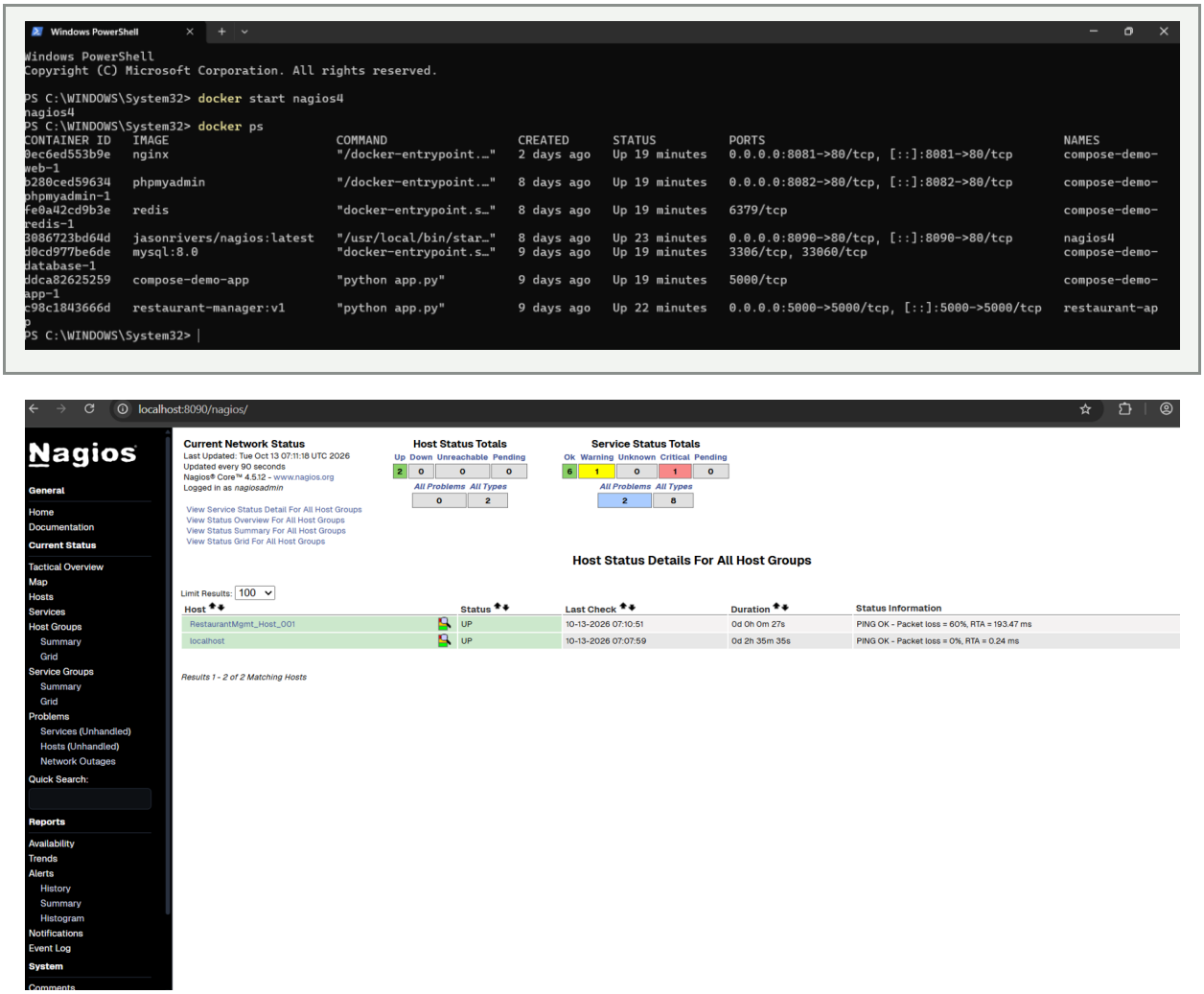


Figure 5.7: Output screenshots of Experiment 7 (Nagios)

5.7.4 Conclusion

Nagios was implemented as a monitoring tool for hosts and services. The system performed continuous checks and displayed the status of the monitored resources, helping identify infrastructure and service issues.

6. Testing

6.1 Automated Unit Tests

The tests use pytest and the Flask test client, so they run without starting a server. They are kept in app/tests. The first checks that GET /health returns HTTP 200, the second checks that GET /api/menu returns a non-empty menu, and a third (test_pages.py) checks that the home page loads.

```
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from app import app

def test_health():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200

def test_menu_api():
    client = app.test_client()
    response = client.get("/api/menu")
    assert response.status_code == 200
    assert len(response.get_json()) > 0
```

The tests are run locally with python -m pytest app/tests, and by GitHub Actions on every push and pull request to main (Section 5.6).

6.2 Container and Service Checks

Check	How it was verified	Expected result
Containerised application	docker run, then http://localhost:5000/health	HTTP 200 with {"status": "ok"}
Compose stack	docker compose ps; http://localhost:8081 and http://localhost:8082	All services running; application through nginx; phpMyAdmin login page
Reverse proxy	Request to nginx on port 8081	Response from the Flask application
Jenkins pipelines	Build Now, Console Output	Finished: SUCCESS
Monitoring	Nagios Current Status, Hosts and Services	Host UP and PING service OK

Table 6.1: Checks performed on the deployed system

6.3 Pipeline Verification

Pipeline verification is done at three levels: the pull request check from GitHub Actions (green run), the Jenkins build triggered by the push (Build, Test and Deploy stages completed), and the board state in Jira (items moved to In Review and Done by the automation rules).

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Menu, cart and ordering	Achieved: category filter, cart, order placement with total calculated on the server (Chapter 4)
Chatbot and admin dashboard	Achieved: Restaurant Assistant answers menu, price and order-status questions; staff manage orders and the menu (Chapter 4)
Jira Scrum planning linked to code	Achieved: backlog and sprint in project SRT; GitHub for Jira connected; automation rules (Section 5.6)
Git and GitHub workflow	Achieved: branches, merges, conflict resolution, reset, revert, restore (Section 5.1)
Docker image and container	Achieved: restaurant-manager:v1 running on port 5000 (Section 5.2)
Docker Compose stack	Achieved: nginx, app, MySQL, Redis and phpMyAdmin from one file (Section 5.3)
Jenkins pipelines and monitoring	Achieved: Declarative and Scripted pipelines, Jenkinsfile from GitHub (Sections 5.4, 5.5); Nagios host and PING service, NSClient++ configured (Section 5.7)

Table 7.1: Objectives and their outcomes

7.2 Observations

- Putting the issue key in the branch name and commit message made the Jira Development panel and the automation rules work without any manual update of the board.
- Running Nagios in a container made the monitoring server easy to start and discard. Because the monitored PC was on a hotspot network whose addresses can change, the host check depended on the current IP address, which shows why stable addressing matters for monitoring.
- The health endpoint makes the application easy to check from Docker, Jenkins and Nagios, and the chatbot reading order status from the same database as the admin dashboard keeps what the customer sees consistent with what the staff enter.

7.4 Limitations

- The chatbot is rule-based, so a question phrased in an unusual way may not be understood.
- The Deploy stage of the Jenkins pipeline uses echo steps, as in the lab experiments, and does not yet start the containers.
- The monitored host is checked with PING; application-level checks such as /health are an optional extension.

8. Conclusion and Future Scope

8.1 Conclusion

The Smart Restaurant Management System was delivered through a complete Agile and DevOps flow. The work was planned in Jira, versioned in Git and GitHub, packaged with Docker and Docker Compose, built and deployed through Jenkins, tested by GitHub Actions and monitored by Nagios. Each of the two members performed part of every experiment and produced evidence of it. The project shows that these practices are most valuable together: each tool takes its input from the previous one, and the issue key, the commit, the pull request, the build and the monitoring check all refer to the same unit of work.

8.2 Future Scope

- Replace the echo steps of the Jenkins Deploy stage with real steps that build the image and run docker compose up -d, with a smoke test and automatic rollback.
- Add application-level monitoring in Nagios with check_http against /health, and add Prometheus and Grafana for metrics.
- Replace the keyword chatbot with a language-model assistant that can take orders in natural language.
- Store the menu and orders in the MySQL service instead of SQLite, and add customer accounts, payment and kitchen display screens.
- Deploy the application to a cloud service or a college server with automatic redeployment from the Jenkins pipeline.

References

1. S. Chacon and B. Straub, Pro Git, 2nd ed. Apress, 2014. https://git-scm.com/book
2. Docker Inc., Docker Documentation. https://docs.docker.com
3. Docker Inc., Docker Compose Documentation. https://docs.docker.com/compose
4. Jenkins Project, Jenkins User Documentation and Pipeline Syntax. https://www.jenkins.io/doc
5. GitHub, GitHub Docs: Actions, Webhooks and Pull Requests. https://docs.github.com
6. Atlassian, Jira Software Documentation and Agile Coach. https://www.atlassian.com/agile

7. K. Schwaber and J. Sutherland, The Scrum Guide. https://scrumguides.org
8. Nagios Enterprises, Nagios Core Documentation. https://www.nagios.org/documentation
9. NSClient++ Documentation. https://nsclient.org
10. Pallets, Flask Documentation. https://flask.palletsprojects.com
11. pytest Documentation. https://docs.pytest.org

Appendix A: Running the Project

A.1 Run locally

```
cd smart-restaurant-system
python -m venv .venv
.venv\Scripts\activate
pip install -r app\requirements.txt
python app\app.py
# open http://localhost:5000/health
```

A.2 Run as a container

```
docker build -t restaurant-manager:v1 .
docker run -d --name restaurant-app -p 5000:5000 restaurant-manager:v1
```

A.3 Run the full stack with Docker Compose

```
cd compose-demo
docker compose up -d
# application:  http://localhost:8081
# phpMyAdmin:   http://localhost:8082  (server database, user root)
docker compose down
```

A.4 Run the tests

```
pip install pytest
python -m pytest app/tests
```

A.5 Start Jenkins and Nagios

```
cd C:\jenkins
java -jar jenkins.war                # http://localhost:8080

docker run --name nagios4 -d -p 8090:80 jasonrivers/nagios:latest   #
http://localhost:8090/nagios/
```

Lab Outcomes

A. LO1 – Apply Agile methodologies and use Jira for sprint planning, task tracking and integrating with the DevOps workflow (Experiment 6).

B. LO2 – Perform version control operations using Git and apply collaboration workflows (Experiment 1).

C. LO3 – Build CI/CD pipelines using Jenkins with Declarative and Scripted syntax, and automate application deployment using GitHub and Jenkins (Experiments 4 and 5).

D. LO4 – Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools (Experiments 2 and 3).

E. LO5 – Implement monitoring of hosts and services using Nagios (Experiment 7).